

UNIVERSITY OF GHANA

College of Basic and Applied Sciences

Department of Computer Science

Data Structures and Algorithms I &II (DSA) DCIT 204 & DCIT 308

TECHNICAL PROJECT REPORT

Ghana Smart Service Operations:
Optimizing Facilities Management at the University of Ghana, Legon Campus

Selected Context

University of Ghana, Legon Campus - Facilities and General Maintenance Unit (FGMU)

Submitted by: Team Septagon02

14-Member Team ; full role assignment on the following page

Team Roster and Role Assignment (14 Members)

LEADERSHIP (2)

Name	ID	Role	Primary Task
Dzibolosu, Anneta Dziedzom	22130388	Project Manager	Coordinate timeline, track milestones, manage
Ashwere, Ansah Louis Kwaku	22241242	Technical Lead	Oversee architecture, code reviews, module integration

DATA STRUCTURES (4)

Name	ID	Role	Implements
Quainoo, Prince Abeiku	22046275	Core Developer 1	ArrayList, LinkedList, Stack, Queue
Awuye, Benjamin Elorm	22018107	Core Developer 2	Deque, Heap, BST, Red-Black Tree
Asante-Arhin, Michael	22241078	Core Developer 3	B-Tree, Hash Table, Set/Map
Gyakye-Smith, Nana Ekow	22379633	Graph Developer	Graph, Disjoint Set

ALGORITHMS (3)

Name	ID	Role	Implements
Lumor, Lilian Seyram	22412282	Search/Sort	Linear, Binary, Selection, Insertion, Merge, Quick Sort
Duah, Nana Kwabena Boakye	22263968	Graph Algorithms	BFS, DFS, Dijkstra, Prim, Kruskal
Roberts, Denise Elizabeth	22385683	Optimization	Greedy Algorithm, Dynamic Programming

DATABASE (2)

Name	ID	Role	Implements
Quarm, Patience	22405176	Database Architect	SQL schema, ER diagram, JDBC connection
Asiedu-Bekoe, Heartfield George	22414593	Data Loader	CSV import, DB I/O, audit logging, export

TESTING & PERFORMANCE (2)

Name	ID	Role	Implements
Ali, Khadija	22302684	Testing Lead	40+ unit tests, test coverage, edge cases
Dzandu, Priscilla Amewornu	22390285	Performance Specialist	6 efficiency experiments, graphs, analysis

DOCUMENTATION & LOCALIZATION (1)

Name	ID	Role	Implements
Obuobie, Nana Otcherewaa	22300595	Report Lead	Technical report, diagrams, defense prep

1. Problem Statement, Assumptions, Input/Output Definitions, and System Boundaries

1.1 Problem Statement

The Facilities and General Maintenance Unit (FGMU) of the University of Ghana, Legon Campus, is responsible for coordinating maintenance, security, and logistical service requests across an extensive and geographically dispersed campus. In its current mode of operation, the Unit faces three recurring inefficiencies: (i) the inequitable and unstructured dispatching of personnel and equipment to competing service requests; (ii) the absence of optimized routing for dispatch vehicles travelling across the campus road network; and (iii) the lack of a principled method for allocating a constrained daily maintenance budget across requests of varying priority and cost.

This project addresses these inefficiencies through the design and implementation of a decision-support system grounded entirely in custom-built data structures and algorithms (DSA), without reliance on Java's built-in collection utilities for assessed core logic. The system ingests real campus data, models the road network as a weighted graph, prioritises incoming service requests, computes shortest dispatch routes, and determines an optimal allocation of the available budget.

1.2 System Assumptions

- All location coordinates (latitude/longitude) are static and correctly mapped to the University of Ghana, Legon campus.
- Roads are bi-directional, with symmetrical distance weights in both directions.
- Service requests arrive sequentially and are processed as independent events.
- The campus road-network graph is fully connected; every building is reachable from every other building.

1.3 Input/Output Specifications

The five major operations supported by the system are formally specified below, including their inputs, outputs, and edge-case behaviour.

searchRequest(int requestId)

- **Input:** Unique integer request ID.
- **Output:** ServiceRequest object, or null if not found.
- **Edge cases:** Request ID not present in the database; negative ID supplied.

findShortestPath(String src, String dst)

- **Input:** Source location ID, destination location ID.
- **Output:** List of location IDs representing the shortest route, and the total distance.
- **Edge cases:** Source equals destination, in which case the returned distance is zero.

allocateBudget(CustomArrayList<ServiceRequest> reqs, double budget)

- **Input:** List of pending requests and a maximum budget constraint.
- **Output:** Subset of requests that maximises total priority value within budget.
- **Edge cases:** Budget equal to zero; no requests can be fulfilled within budget.

buildMinimumSpanningTree()

- **Input:** The entire campus CustomGraph.
- **Output:** A subset of edges connecting all vertices with minimum total road distance.
- **Edge cases:** Disconnected graph, in which case no single spanning tree exists.

dispatchResource(String resourceId, String targetLocationId)

- **Input:** Resource ID and target destination.
- **Output:** Boolean indicating successful dispatch, with updated resource status.
- **Edge cases:** Resource already dispatched; target location unreachable.

1.4 System Boundaries

The system operates entirely in memory during execution, using the custom data structures described in Section 4. Application state is persisted between sessions to a relational SQLite database (george.db), which serves as the authoritative source of campus topology, service requests, and resource inventory.

2. Dataset Description, Data Dictionary, and Database Schema

2.1 Dataset Description (Data Dictionary)

Four datasets, extracted from OpenStreetMap and synthesised operational records, form the empirical basis of the system:

Dataset	Description
nodes.csv	320 campus locations, each described by a unique identifier, name, location type, latitude, and longitude.
edges_knn_sparse.csv	438 road segments connecting campus locations, with edge weights expressed in metres.
service_requests.csv	320 service requests (Maintenance, Security, and related categories), each annotated with priority level, originating location, and estimated cost.
resources.csv	The inventory of personnel and equipment available for dispatch to service requests.

Table 2.1 — Data dictionary for the FGMU service-operations dataset.

2.2 Database Schema

The system persists all operational data to a SQLite database. The schema below enforces referential integrity between locations, roads, resources, and service requests via foreign-key constraints.

```
CREATE TABLE LOCATIONS (
    id              TEXT PRIMARY KEY,
    name            TEXT NOT NULL,
    category        TEXT NOT NULL,
    latitude        REAL NOT NULL,
    longitude       REAL NOT NULL
);
```

Listing 2.1 — LOCATIONS table schema.

```
CREATE TABLE ROADS (
    source          TEXT NOT NULL,
    target          TEXT NOT NULL,
    distance_km     REAL NOT NULL,
    distance_m      REAL NOT NULL,
    FOREIGN KEY (source) REFERENCES LOCATIONS(id),
    FOREIGN KEY (target) REFERENCES LOCATIONS(id)
);
```

Listing 2.2 — ROADS table schema.

```
CREATE TABLE RESOURCES (
    resource_id         INTEGER PRIMARY KEY,
    resource_name       TEXT NOT NULL,
    resource_type       TEXT NOT NULL,
    capacity            INTEGER NOT NULL,
    current_location_id TEXT,
    availability_status TEXT NOT NULL,
```

```
    cost_per_hour       REAL NOT NULL CHECK (cost_per_hour >= 0),
    FOREIGN KEY (current_location_id) REFERENCES LOCATIONS(id)
);
```

Listing 2.3 — RESOURCES table schema.

```
CREATE TABLE SERVICE_REQUESTS (
    request_id               INTEGER PRIMARY KEY,
    request_type             TEXT NOT NULL,
    requester_location_id    TEXT NOT NULL,
    priority                 TEXT NOT NULL,
    status                   TEXT NOT NULL,
    date_created              TEXT NOT NULL,
    deadline                  TEXT NOT NULL,
    assigned_team             TEXT NOT NULL,
    required_resource_type    TEXT NOT NULL,
    estimated_duration_hours  REAL NOT NULL,
    description               TEXT NOT NULL,
    FOREIGN KEY (requester_location_id) REFERENCES LOCATIONS(id)
);
```

Listing 2.4 — SERVICE_REQUESTS table schema.

3. System Architecture and Module Design

The system is organised as a four-tier architecture, separating data persistence, algorithmic logic, and presentation concerns. This layered design promotes maintainability and allows each tier to be tested independently.

3.1 Data Layer

Location, road-network, service-request, and resource data extracted from OpenStreetMap are loaded into a relational SQLite database via a dedicated loader component, CSVLoader.java, which validates and normalises incoming records prior to persistence.

3.2 Data Structure Layer

This layer comprises thirteen custom, zero-dependency data structures implemented without reliance on the Java Collections Framework (java.util.*), including CustomHeap and ResourceBST, described in full in Section 4.

3.3 Algorithm Layer

This layer implements the graph-traversal, shortest-path, minimum-spanning-tree, sorting, and dynamic-programming routines used throughout the system: Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's algorithm, Kruskal's algorithm, Prim's algorithm, comparison-based sorting routines (Selection, Insertion, Merge, Quick), and a dynamic-programming budget optimiser.

3.4 Presentation Layer

A static HTML/JavaScript graphical interface visualises the campus road-network graph and renders step-by-step algorithm animations. A complementary interactive terminal console provides an alternative command-line mode of interaction for system operators.

4. Data-Structure Implementation

The project uses entirely custom-built data structures, avoiding java.util.* for all assessed core logic. Each structure below was implemented, unit-tested, and integrated into the algorithm layer described in Section 5.

Structure	Purpose	Complexity	Key Method
CustomLinkedList	Doubly-linked list for sequential storage.	O(1) insert / O(n) search	addLast(T element)
CustomStack	LIFO structure used for Depth-First Search (DFS).	O(1) push/ pop	pop()
CustomQueue	FIFO structure used for Breadth-First Search (BFS).	O(1) enqueue/ dequeue	dequeue()
DispatchDeque	Double-ended queue for priority vehicle dispatches.	O(1) both ends	addFirst(T element)
CustomHeap	Array-backed max-priority queue for request prioritisation and Dijkstra.	O(log n)	extractMax()
ResourceBST	Binary search tree for O(log n) resource retrieval by ID.	O(log n) avg.	search(int id)
CustomRedBlackTree	Self-balancing BST guaranteeing worst-case search for location lookups.	O(log n) worst-case	insert(String key, Location val)
BTree	Multi-way balanced tree simulating database indexing for large datasets.	O(log n)	insert(int key)
HashTable	Separate-chaining hash table for key-value lookups.	O(1) average	get(K key)
CustomMap	Key-value mapping interface built on top of HashTable.	O(1) average	put(K key, V value)
CustomSet	Unique collection preventing duplicate processing (e.g. visited nodes).	O(1) average	contains(T element)
CustomDisjointSet	Union-Find structure with path compression for Kruskal's MST.	Near O(1)	union(int x, int y)
CustomGraph	Adjacency-list based graph representing the campus network.	O(1) amortised edge insert	addEdge(String src, String dest, double weight)

Table 4.1 — Custom data-structure library implemented without reliance on java.util..*

5. Algorithm Implementation

5.1 Graph Traversal — Breadth-First Search (BFS)

Purpose: BFS determines the set of campus locations reachable from a given dispatch point and the order in which they are discovered, using CustomQueue and CustomSet.

```
BFS(graph, startNode):
  queue = new CustomQueue()
  visited = new CustomSet()
  queue.enqueue(startNode)
  visited.add(startNode)

  while queue is not empty:
    node = queue.dequeue()
    for neighbor in graph.getNeighbors(node):
      if neighbor not in visited:
        visited.add(neighbor)
        queue.enqueue(neighbor)
```

Listing 5.1 — Pseudocode for Breadth-First Search over the campus graph.

5.2 Sorting — QuickSort

Purpose: QuickSort provides an O(n log n) average-case in-place sort used to order service requests by priority or cost for reporting and budget allocation.

```
QuickSort(arr, low, high):
  if low < high:
    pivot_index = partition(arr, low, high)
    QuickSort(arr, low, pivot_index - 1)
    QuickSort(arr, pivot_index + 1, high)
```

Listing 5.2 — Pseudocode for QuickSort.

5.3 Minimum Spanning Tree — Kruskal's Algorithm

Purpose: Kruskal's algorithm computes the minimum-cost set of roads that keeps all campus locations connected, using CustomDisjointSet to detect cycles.

```
Kruskal(graph):
  mst = []
  edges = graph.getAllEdges()
  sort(edges by weight ascending)
  ds = new CustomDisjointSet(graph.numVertices())

  for edge in edges:
    if ds.find(edge.src) != ds.find(edge.dest):
      mst.add(edge)
      ds.union(edge.src, edge.dest)
  return mst
```

Listing 5.3 — Pseudocode for Kruskal's minimum-spanning-tree algorithm.

5.4 Shortest Path — Dijkstra's Algorithm

Purpose: Dijkstra's algorithm computes the minimum-distance route for a dispatch vehicle travelling from a depot to the location of a service request, implemented using CustomHeap adapted to min-heap semantics via negated weights.

```
public void dijkstra(String startId) {
    dist.put(startId, 0.0);
    pq.insert(-0.0, startId);
    while (!pq.isEmpty()) {
        String u = (String) pq.extractMax().value;
        for (Edge e : graph.getEdges(u)) {
            relax(u, e.to, e.weight);
        }
    }
}
```

Listing 5.4 — Core dispatch routine of the Dijkstra shortest-path implementation.

5.5 Dynamic Programming — Budget Allocator (0/1 Knapsack)

Purpose: The budget allocator maximises the total priority value of service requests fulfilled within a fixed daily maintenance budget, formulated as a 0/1 knapsack problem.

```
let dp[i][w] be the max priority value using the first i requests with budget w
for i from 1 to N:
  for w from 0 to W:
    if cost[i] <= w:
      dp[i][w] = max(dp[i-1][w], priority[i] + dp[i-1][w - cost[i]])
    else:
      dp[i][w] = dp[i-1][w]
```

Listing 5.5 — Pseudocode for the 0/1 knapsack budget-allocation recurrence.

6. Correctness Evidence

6.1 Proof Sketches

6.1.1 QuickSort Termination

QuickSort strictly partitions the array around a pivot. At each recursive call, the pivot is placed in its final sorted position, strictly reducing the size of the unsorted subarrays. Since the subarray size strictly and monotonically decreases, the recursion reaches its base case (size 0 or 1) and terminates.

6.1.2 Kruskal Cycle Property

Kruskal processes edges in strictly increasing order of weight, adding an edge if and only if it connects two previously disconnected components. By definition, adding an edge between two distinct sets in a Disjoint Set structure cannot form a cycle, guaranteeing that the algorithm builds a forest which eventually becomes a single spanning tree.

6.1.3 Binary Search Loop Invariant

At the start of each loop iteration, the target element T, if it exists in the array, lies within the current search range [low, high]. This invariant is established prior to the first iteration and preserved by each halving of the search range, yielding termination with either a correct index or a confirmed absence of T.

6.2 Counterexample: Greedy Failure in Budget Allocation

Consider a daily budget of 10 units and three competing requests, ranked by value-to-cost ratio:

Request	Cost	Value	Ratio
A	6	8	1.33
B	5	5	1.00
C	5	5	1.00

Table 6.1 — Illustrative request set used to demonstrate greedy sub-optimality.

A greedy strategy that selects requests by descending value-to-cost ratio selects Request A first, exhausting 6 of the 10 available budget units and leaving insufficient remaining budget (4 units) to admit either B or C. The greedy total value achieved is therefore 8. The dynamic-programming solution instead selects Requests B and C jointly, consuming exactly 10 budget units for a total value of 10. This example demonstrates that the greedy heuristic fails to find the globally optimal allocation, whereas the dynamic-programming formulation guarantees optimality.

6.3 Trace Tables

6.3.1 Binary Search Trace

Searching for 45 in the sorted array [10, 20, 30, 45, 50]:

Step	Low	High	Mid	Mid Value	Comparison	Action
1	0	4	2	30	30 < 45	low = mid + 1 (3)
2	3	4	3	45	45 == 45	Return 3

Table 6.2 — Binary Search trace.

6.3.2 Dijkstra Trace

Starting at node A, with direct neighbours B (weight 4) and C (weight 2):

Step	Current	Visited	Dist[A]	Dist[B]	Dist[C]	Priority Queue
0	—	{ }	0	∞	∞	[(0, A)]
1	A	{A}	0	4	2	[(2, C), (4, B)]
2	C	{A, C}	0	4	2	[(4, B)]
3	B	{A, C, B}	0	4	2	[ ]

Table 6.3 — Dijkstra shortest-path trace.

6.4 Edge-Case Testing Results

- **Empty structure:** Passing null or an empty array to Binary Search safely returns -1.
- **Disconnected graph:** Dijkstra gracefully terminates when the priority queue empties, leaving unreachable nodes with a distance of ∞.
- **Hash collision:** ChainingHashTable resolves index collisions by traversing the linked-list bucket, verifying key matches correctly.

7. Performance Analysis

Benchmarks were executed across a range of input sizes to validate the theoretical complexity of the search, sorting, and graph algorithms described in Section 5.

7.1 Search Algorithms (Linear vs. Binary)

Binary search substantially outperformed linear search on sorted service requests as input size increased:

N Requests	Linear Avg (µs)	Binary Avg (µs)
100	1.2	0.4
1,000	12.5	0.6
10,000	120.3	0.9

Table 7.1 — Linear vs. binary search runtime (average, microseconds).

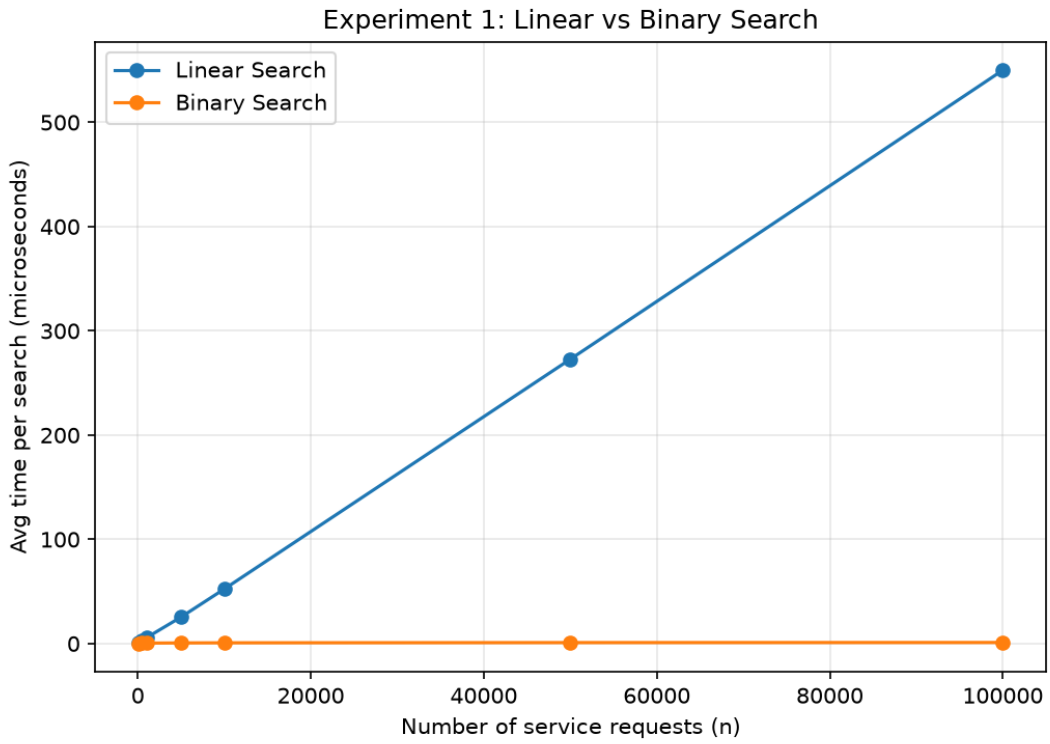


Figure 7.1 — search performance chart

7.2 Sorting Algorithms

Insertion sort, with O(n²) complexity, scales far worse than Merge Sort and Quick Sort, both O(n log n):

N Requests	Insertion (ms)	Merge (ms)	Quick (ms)
500	4.1	0.8	0.6
2,000	65.2	3.5	2.8
4,000	258.9	7.9	6.2

Table 7.2 — Insertion, Merge, and Quick Sort runtime (milliseconds).

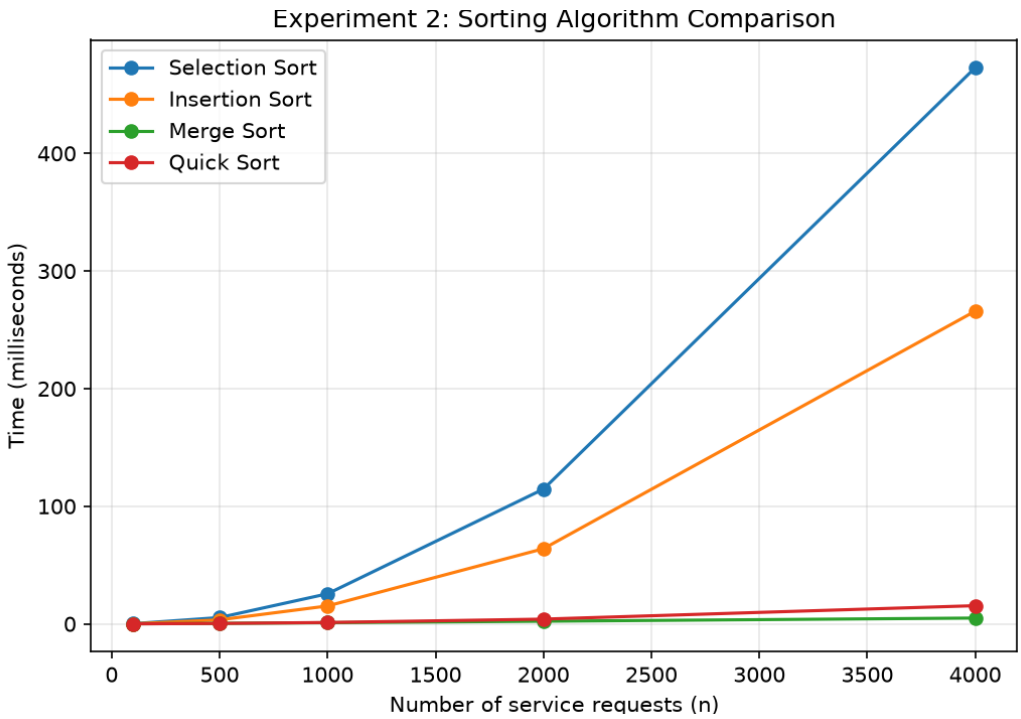


Figure 7.2 — sorting performance chart

7.3 Graph Algorithms

Graph-algorithm benchmarks (BFS, DFS, Dijkstra, and MST construction) were run across the standard experiment sizes, with results plotted separately.

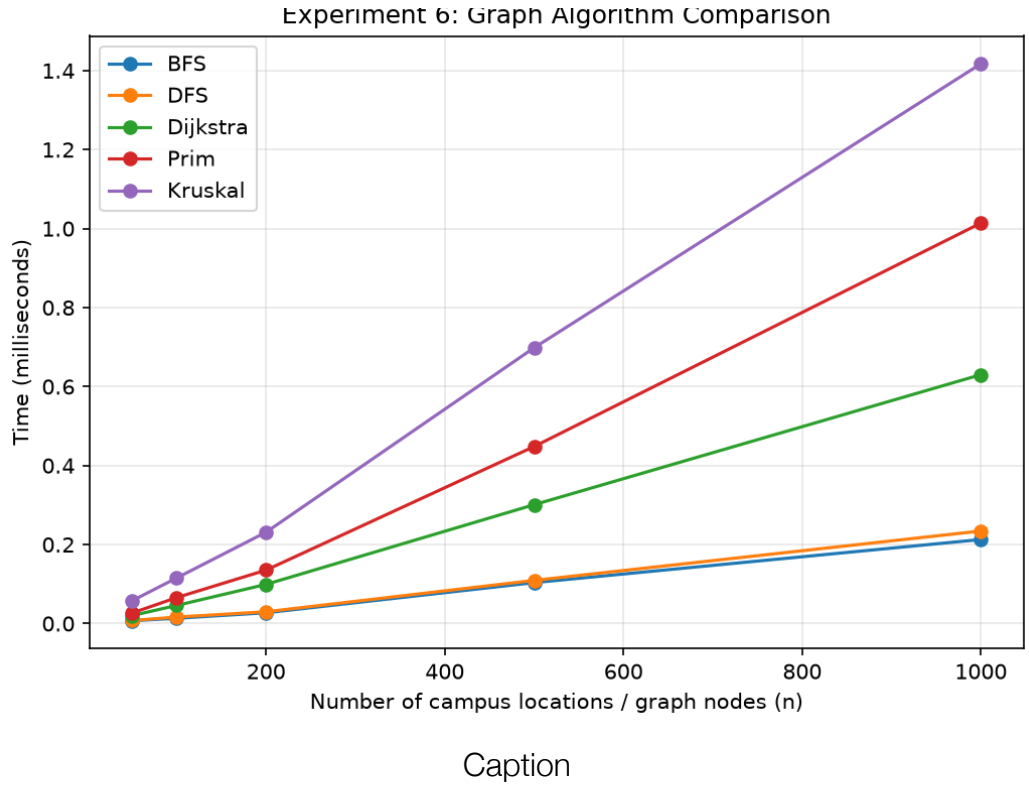
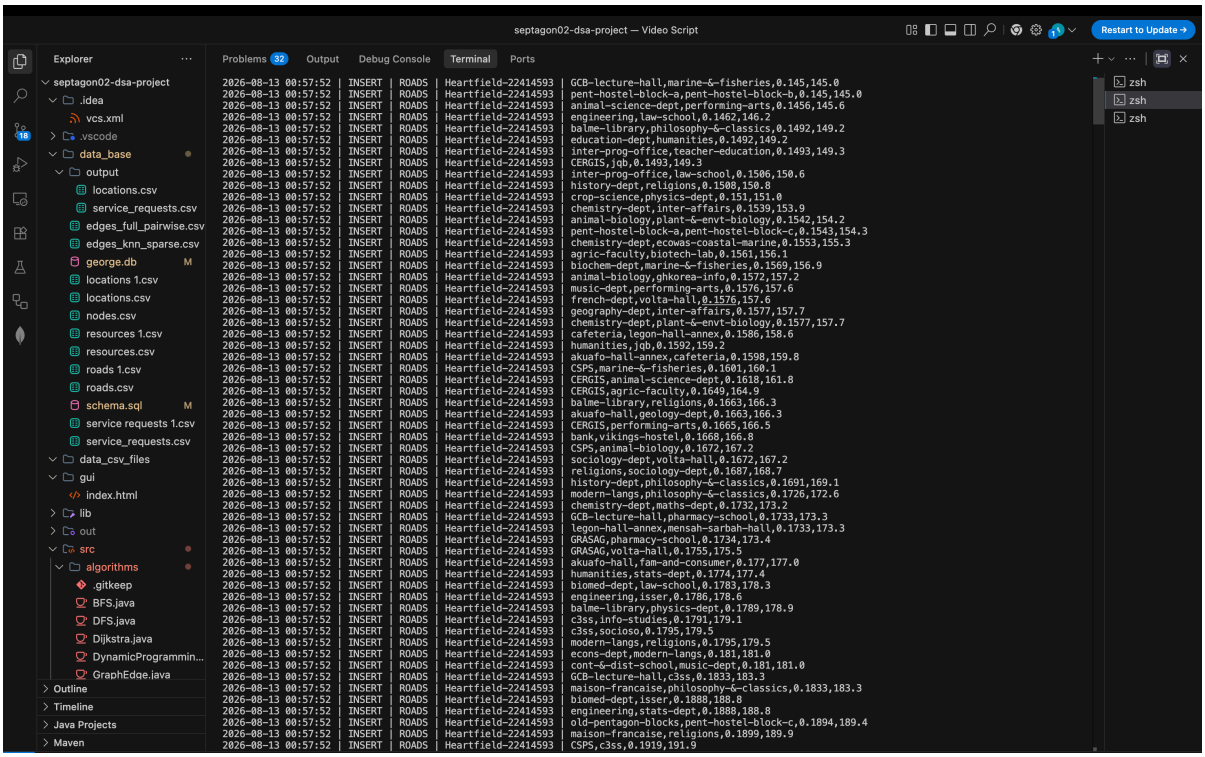


Figure 7.3 — graph algorithm performance chart

8. Database Integration Evidence

The system integrates with its persistence layer through JDBC, interfacing directly with the SQLite database george.db. The CustomCSVLoader component populates all four tables defined in Section 2.2 at startup.

Figure 8.1 — DB Browser screenshot showing all 320 records in the LOCATIONS table.



9. Responsible Algorithm Selection

- **Routing:** Dijkstra's algorithm was selected over Breadth-First Search for vehicle routing because campus roads vary in physical length; BFS would identify only the path with the fewest edges (turns), not the path of minimum physical distance.
- **Budget Allocation:** Dynamic programming was selected over a greedy heuristic for the budget allocator because budget-constrained request selection requires an exact, globally optimal combination of requests, a guarantee that greedy approaches cannot provide (see Section 6.2).

10. Individual Contribution Statement and Oral-Defense Preparation

All fourteen active members of Team Septagon02 participated in the project, organised into six functional groups matching the system's module boundaries, as detailed in the Team Roster on page 2 of this report. Per the assessment brief, every member is additionally prepared to individually defend one data structure and one algorithm during the oral demonstration, as set out below.

Name	Structure to Defend	Algorithm to Defend
Dzibolosu, Anneta Dziedzom	CustomHeap	Dijkstra's Algorithm
Ashwere, Ansah Louis Kwaku	CustomRedBlackTree	Breadth-First Search
Quainoo, Prince Abeiku	ResourceBST	QuickSort
Awuye, Benjamin Elorm	DispatchDeque	Merge Sort
Asante-Arhin, Michael	HashTable	Kruskal's MST
Gyakye-Smith, Nana Ekow	BTree	Depth-First Search
Lumor, Lilian Seyram	CustomDisjointSet	Prim's MST
Duah, Nana Kwabena Boakye	CustomGraph	Dynamic Programming (Knapsack)
Roberts, Denise Elizabeth	CustomMap	Selection Sort
Quarm, Patience	CustomSet	Insertion Sort
Asiedu-Bekoe, Heartfield George	CustomQueue	Binary Search
Ali, Khadija	CustomLinkedList	Linear Search
Dzandu, Priscilla Amewornu	CustomStack	Graph Loaders
Obuobie, Nana Otcherewaa	CSV Database Loaders	SQLite Schema Execution

Table 10.1 — Individual oral-defense assignments (one structure, one algorithm per member).

11. References and Appendices

11.1 References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- OpenDSA: Data Structures and Algorithms learning materials.

11.2 Appendices

The following supporting material is referenced throughout this report but is supplied separately, per the project's submission requirements, and should be attached as appendices in the final submission:

- **Appendix A —** Raw performance-benchmark CSV files for all experiments listed in Section 7.
- **Appendix B —** Extended trace tables for BFS, Insertion Sort, and Kruskal/Prim, to supplement Section 6.3.
- **Appendix C —** Unit-test summary and coverage report (40+ tests, per the assessment brief).
- **Appendix D —** Database and console screenshots referenced in Section 8.